

Relatório Aula 07 - Expressões, Atribuição e Controle

4 - Slide:

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

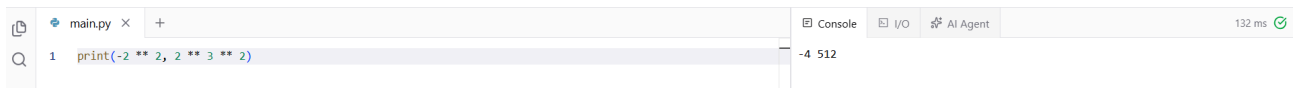
6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

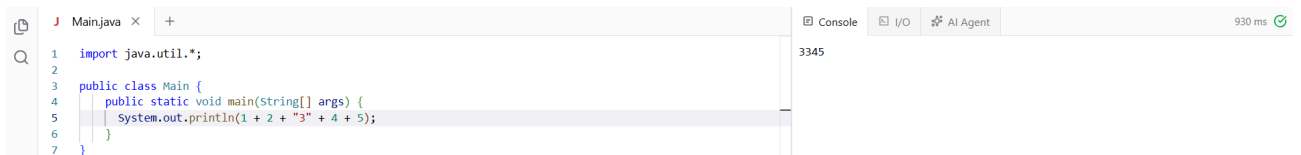
As respostas aparecem ao longo da aula · Sebesta Cap. 7

Nota: O quiz evidencia como diferentes linguagens tratam sintaxes semelhantes de formas opostas:

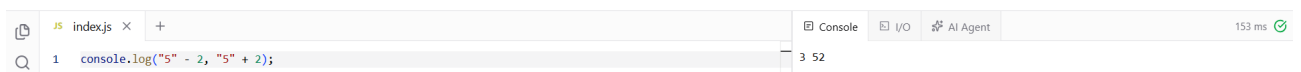
- Python ($-2^{**}2e2^{**}3^{**}2$): A potenciação tem precedência sobre o menos unário ($-2^{**}2$ vira $-(2^{**}2) = -4$) e possui associatividade à direita ($2^{**}(3^{**}2) = 2^{**}9 = 512$).



- Java ($1+2+"3"+4+5$): Avaliado da esquerda para a direita; avalia $1+2$ (3), concatena com "3" (resultando na String "33"), e depois converte os demais operandos para String por causa da coerção ("3345").



- JavaScript ($"5"-2$ e $"5"+2$): O operador - força a coerção para número ($5-2 = 3$), enquanto o operador + prioriza a concatenação de texto ("52").



- C ($3 > 2 > 1$): Avalia da esquerda para a direita; $(3 > 2)$ resulta no booleano/inteiro 1. Em seguida, avalia $1 > 1$, retornando 0 (falso).

```
C Main.c x +
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

Console I/O AI Agent 177 ms

0

- Python (x or "padrão" e $3 > 2 > 1$): O operador or retorna o primeiro valor verdadeiro (como $x=0$ é falsy, devolve "padrão"). Já o Python encadeia comparações automaticamente como $(3 > 2)$ and $(2 > 1)$, retornando True.

```
main.py x +
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
3
```

Console I/O AI Agent 130 ms

padrão True

- JavaScript ($0 == ""$ vs $0 == "0"$ vs $0 === "0"$): Demonstra a não-transitividade e as inconsistências da comparação fraca ($==$) em comparação com a checagem estrita ($===$).

```
JS index.js x +
1 console.log(0 == "", "" == "0", 0 == "0");
2
```

Console I/O AI Agent 135 ms

true false true

5 - Slide:

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

a + b * c com a=3, b=4, c=5

da esquerda para a direita

(3 + 4) * 5 = 35

da direita para a esquerda

3 + (4 * 5) = 23

com precedência (* antes de +)

3 + 4 * 5 = 23

APL: sempre da direita para a esquerda (p. 304)

3 x 4 + 5 = 27

Sebesta, 11ª ed. · §7.2 e §7.2.1 · p. 300-305

Nota: A modelagem de expressões aritméticas exige escolhas de arquitetura da linguagem:

1. Precedência: Agrupa operadores conforme a prioridade (ex: * antes de +).
2. Associatividade: Agrupa operadores de mesma precedência à esquerda ou à direita.
3. Ordem de avaliação de operandos: Define se $f(a)$ roda antes de $f(b)$ em $f(a) + f(b)$.
4. Efeitos colaterais: Restrições quando a avaliação de uma função altera o estado de variáveis externas.
5. Sobrecarga de operadores: Capacidade de redefinir símbolos (como +) para novos tipos.
6. Mistura de tipos: Define regras de compatibilidade, coerção implícita e conversões.

Sintaxes exóticas: O Lisp usa notação prefixa baseada em parênteses (+ a (* b c)) eliminando a necessidade de regras de precedência. O APL avalia estritamente da direita para a esquerda.

6 - Slide:

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	–
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)

Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran),
++, --, menos unário
Visual Basic: ^ à esquerda!

Excel

= -2^2 dá 4: lá o menos unário vence o ^.

Na dúvida, use **parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

Compreendi que diferentes linguagens aplicam regras de precedência e associatividade distintas. A potenciação em Python é associativa à direita, enquanto linguagens como o Visual Basic associam à esquerda.

Nota: Tabela comparativa e pegadinhas sintáticas:

- Potenciação (**): Python, JS, Ruby e Fortran usam associatividade à direita ($2^{\{3^2\}} = 512$). No Visual Basic a associação é à esquerda.
- Divisão de inteiros (-7 / 2): Em Python (//) arredonda para baixo (piso), resultando em -4. Em C e Java o resultado é truncado em direção ao zero, resultando em -3. Em JS resulta em número real (-3.5).
- Módulo (-7 % 2): Python mantém o sinal do divisor (1), enquanto C, Java e JS mantêm o sinal do dividendo (-1).
- Regra de Ouro: Na dúvida, o uso de parênteses explícitos garante que o código seja interpretado corretamente pelo compilador e compreendido com facilidade por outros desenvolvedores.

7 - Slide:

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

Compreendi que a ordem de avaliação dos operandos e os efeitos colaterais podem fazer o mesmo código retornar valores totalmente diferentes dependendo da linguagem (como 8 em Java e 20 em C/Go).

Nota: Em $a = a + \text{fun1}()$, onde $\text{fun1}()$ altera o valor global de a para 17 e retorna 3:

- Java: Garante avaliação estrita da esquerda para a direita. O valor do primeiro a (5) é lido antes de chamar $\text{fun1}()$, resultando em $5 + 3 = 8$.
- C (gcc 14) e Go: A ordem de leitura dos operandos em relação às chamadas de função não é especificada pelo padrão. Se o compilador invocar $\text{fun1}()$ primeiro, a é atualizado para 17 e a expressão vira $17 + 3 = 20$.
- Transparência Referencial: Um programa possui transparência referencial se qualquer expressão puder ser substituída pelo seu valor sem alterar o comportamento do programa. A ausência de efeitos colaterais é indispensável para otimizações do compilador.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("'" + 'a' + 1);     // a1
x.add(y).multiply(tres);               // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):           # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):                # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)          # Vetor(4, 6)
print(v * 3)                           # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Compreendi que algumas linguagens permitem sobrecarregar operadores de forma customizada, enquanto outras proíbem essa prática para evitar ambiguidades ou complexidade excessiva.

Nota: A sobrecarga reutiliza o mesmo operador para contextos variados:

- Vantagens vs Risco: Em C++, o operador & serve tanto para "E bit a bit" quanto para "Endereço de", gerando ambiguidades.
- Implementação em Python: Realizada por meio de métodos especiais dunder (como `__add__` e `__mul__`), permitindo operações aritméticas intuitivas entre instâncias de classes personalizadas (ex: vetores).
- Posição do Java: O Java proíbe a sobrecarga definida pelo usuário para manter a simplicidade da linguagem, oferecendo o operador + sobrecarregado apenas nativamente (soma e concatenação de textos). Tipos numéricos de alta precisão (BigDecimal) exigem chamadas explícitas de métodos como `.add()` e `.multiply()`.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;          // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não : float64(i)	erro de compilação
Rust	não : i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. · §7.4 · p. 310-313 · exemplos/04_conversoes

Compreendi que as conversões de tipos podem ser automáticas (coerção/alargamento) ou explícitas (cast/estreitamento), e que a coerção implícita pode esconder erros de precisão ou de lógica.

Nota: A conversão de tipos varia entre abordagens permissivas e estritas:

- **Alargamento vs Estreitamento:** Alargamento converte tipos menores para maiores (ex: int para float), geralmente sem perda substancial, enquanto estreitamento descarta precisão (ex: de double 3.99 para int 3).
- **Coerções Silenciosas:** No Java, converter um int grande para float pode perder bits de precisão sem emitir aviso do compilador.
- **Linguagens Estritamente Tipadas:** Go e Rust não fazem coerção implícita entre tipos incompatíveis (ex: somar int32 com int64 ou misturar int e float gera erro de compilação).
- **ML e F#:** Não possuem coerção e exigem absoluta identidade de tipos em qualquer operação.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaNaN"
```



```
0 == ""           // true
"" == "0"         // false
0 == "0"          // true  (não é transitivo!)
0 === "0"         // false  (=== não converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true
```



```
md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que **não** fazem coerção.

Sebesta, 11ª ed. · §7.4.1 Coerção em expressões · p. 311-312 · exemplos/04_conversoes

Compreendi que a coerção implícita em linguagens fracamente tipadas como JavaScript e PHP produz efeitos colaterais bizarros. Usar operadores de igualdade estrita (===) é indispensável.

Nota: Exemplos práticos de comportamentos perigosos (type juggling):

- JavaScript: "5" - 2 é 3, mas "5" + 2 resulta em "52". Além disso, expressões como true + 1 resultam em 2 e [] + {} geram "[object Object]".
- PHP e a armadilha do ==: Comparações fracos analisam Strings que lembram notação científica (ex: "0e123" == "0e456") convertendo ambas para o número 0, tornando a comparação verdadeira (true).
- Solução: Utilizar obrigatoriamente os operadores de comparação estrita === e !== para comparar valor e tipo sem coerção automática.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0  0.0  ""  []  {}  None
# contam como VERDADEIRO:
# "0"  [0]  " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. · §7.5 · p. 313-315 · exemplos/05_booleanas

Compreendi que expressões relacionais e booleanas variam na forma como lidam com o encadeamento de comparações e na avaliação de tipos não-booleans em estruturas condicionais.

Nota: Diferenças no tratamento booleano entre linguagens:

- C: Não tinha tipo booleano nativo original; qualquer valor diferente de zero é considerado verdadeiro no if. A expressão `3 > 2 > 1` falha porque `(3 > 2)` vira 1, e `1 > 1` resulta em falso (0).
- Python: Suporta encadeamento sintático direto (`a > b > c` equivale a `a > b and b > c`). O Python avalia valores falsy (0, 0.0, None, "", []) e valores truthy.
- Segurança no Tipo: Linguagens modernas como Java, C#, Go e Rust exigem estritamente expressões booleanas puras dentro do if (ex: `if (x != 0)`), impedindo que inteiros sejam usados como condições.

Avaliação em curto-circuito

Java - && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;          // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

JavaScript - ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // undefined, sem erro

const qtd = 0;
qtd || 10              // 10  (o 0 se perdeu!)
qtd ?? 10              // 0   (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em $(a > b) \ || \ (b++ \ / \ 3)$, o $b++$ só acontece quando $a <= b$. Em Python, 0 or "padrão" devolve "padrão": and e or devolvem um dos operandos.

Sebesta, 11ª ed. · §7.6 · p. 315-317 · exemplos/05_booleans

Compreendi que a avaliação em curto-circuito interrompe a execução de expressões condicionais assim que o resultado final é determinado, o que ajuda a prevenir erros de execução.

Nota: Aplicações e cuidados na avaliação otimizada:

- **Mecanismo de Segurança:** Em Java, o operador `&&` faz curto-circuito; na expressão `s != null && s.length() > 0`, se `s` for nulo, a segunda parte não é avaliada, evitando um erro de `NullPointerException`. Já o operador bitwise `&` avalia ambos os lados e causaria a exceção.
- **Navegação Segura no JS:** O operador de encadeamento opcional `?.` evita erros de leitura de propriedades em objetos nulos, enquanto o operador de coalescência nula `??` substitui apenas `null` ou `undefined` (ao contrário do `||`, que substitui também o número `0` e `""`).
- **Perigo:** Expressões com efeitos colaterais colocados no segundo termo de um curto-circuito (ex: `(a > b) || (b++)`) podem não ser executadas.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

Compreendi como linguagens diferentes encaram sentenças de atribuição — algumas tratando a atribuição como uma expressão que devolve valor e outras tratando-a estritamente como um comando.

Nota: Diferenças entre paradigma imperativo, moderno e funcional:

- Atribuição como Expressão (C/C++/JS): Atribuição retorna valor. Permite sintaxes como `a = b = (c = 10)`, mas abre brecha para bugs graves como digitar `if (x = y)` (atribuição acidental) no lugar de `if (x == y)`.
- Atribuições Seguras (Python/Go/Rust): Python e Go tratam a atribuição estritamente como instrução. Para evitar bugs, `if (x = y)` não compila. O Python introduziu o operador morsa `:=` (Python 3.8+) para permitir atribuição expressiva explícita.
- Atribuição Múltipla: Permite trocar variáveis sem uma variável temporária intermediária (`a, b = b, a`).
- Linguagens Funcionais: Em ML e F#, os bindings (`val` e `let`) criam associações imutáveis.

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);   // 6
```

C · coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Sebesta, 11ª ed. · §7.8 · p. 321-322 · exemplos/06_atribuicao

Compreendi que atribuições de modo misto podem esconder conversões implícitas e gerar perda de precisão ou resultados zerados devido à ordem das operações.

Nota: Perigos em atribuições mistas e coerções automáticas:

- Cast Oculto no Java: Embora `int i = 3.7;` gere erro de compilação por exigir estreitamento explícito, o operador composto `i += 3.7` realiza um cast embutido sem avisos, convertendo o resultado para inteiro.
- Coerção Permissiva no C: C aceita atribuições de tipos incompatíveis silenciosamente (ex: `int i = 3.7` vira 3; `char c = 300` estoura e vira 44; `unsigned int u = -1` resulta em 4294967295).
- Armadilha Clássica: `double taxa = acertos / total * 100;` Se `acertos` e `total` forem inteiros, a divisão é feita inteiramente antes de ser promovida a double. Assim, `7 / 10` resulta em 0, e `0 * 100` resulta em 0.0.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
  x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. · §8.1 · p. 328-330

Compreendi o Teorema de Böhm e Jacopini, que prova que qualquer algoritmo pode ser escrito apenas com sequência, seleção e repetição, tornando o uso de goto totalmente desnecessário.

Nota: Fundamentação teórica da Programação Estruturada:

- O Teorema (1966): Qualquer fluxo de controle computável pode ser construído combinando três estruturas fundamentais: Sequência, Seleção (if/else) e Iteração/Laço Lógico (while).
- Por que existem tantas estruturas? Comandos adicionais como for, switch, do-while e foreach existem por motivos de facilidade de escrita (açúcar sintático) e expressividade.
- O Trade-off da Legibilidade: Linguagens muito extensas ou cheias de atalhos podem dificultar a leitura de códigos feitos por outros programadores.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };           // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                       # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

Compreendi a ambiguidade clássica do "else pendente" nas linguagens imperativas e como a sintaxe da linguagem influencia diretamente a prevenção de erros de leitura.

Nota: O problema do Dangling Else:

- A Regra Geral (C/C++/Java/C#): O else conecta-se sempre ao if mais próximo que não possua um else, ignorando completamente a indentação do código. No exemplo, o else pertence ao segundo if, fazendo com que o programa imprima 1 em vez de não imprimir nada.
- Prevenção por Bloco Obrigatório: Linguagens como Go, Rust e Perl eliminam a dúvida exigindo o uso de chaves {} em qualquer estrutura condicional.
- Seletores como Expressões: Linguagens modernas (Rust e Python) permitem usar expressões condicionais diretamente no lado direito da atribuição de forma limpa (ex: let y = if x > 0 { x } else { 2 * x };

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
    case 1:
    case 3: odd += 1;
    case 2:
    case 4: even += 1;
    default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
    case SAB, DOM -> "fim de semana";
    case SEG, TER, QUA,
        QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
    i32::MIN..=-1 => "negativo",
    0 => "zero",
    1..=9 => "um dígito",
    _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. · §8.2.2 · p. 334-341 · exemplos/07_selecao

Compreendi que a seleção múltipla evoluiu do perigoso switch em C (que exige break manual) para construções mais seguras e exaustivas como o match e o switch como expressão.

Nota: Evolução do desvio múltiplo:

- C e o comportamento Fall-through: Se cada case não terminar com a instrução break, a execução "cai" e executa os blocos dos casos seguintes de forma descontrolada.
- Java 14+ (Switch Expression): Permite o uso da sintaxe de seta `->`, elimina a necessidade de break, impede o fall-through indesejado e retorna valores diretamente.
- Rust e a Exaustividade: O operador match exige a cobertura de todas as possibilidades possíveis do tipo analisado. A omissão do caso padrão (`_`) impede o código de compilar.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. · §8.3.1 · p. 342-345 · exemplos/08_iteracao

Compreendi que o funcionamento do laço for varia profundamente: em C ele é uma estrutura aberta sobre a variável de controle, enquanto em Python ele funciona como um iterador de sequências.

Nota: Projetos de laços contados:

- C (Laço Lógico Disfarçado): A variável de controle pode ser alterada livremente dentro do corpo do laço, alterando o fluxo dos passos.
- Python (Iterador de Sequência): O laço for percorre um objeto iterável. Tentar alterar a variável de controle *i* manualmente dentro do corpo não afeta o valor fornecido pela função `range()` na próxima iteração.
- Questões de Projeto: O projeto do laço deve definir: Qual o escopo da variável? Os limites da iteração são avaliados uma única vez ou a cada volta? A variável pode ser modificada internamente?

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + "," + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

Compreendi os meandros do uso de laços lógicos e iteradores, desde loops infinitos acidentais até recursos avançados como o break rotulado em laços aninhados.

Nota: Armadilhas e estratégias de iteração:

- O Bug do Zune (2008): Um laço while em ano bissexto não alterava a variável de decremento de dias caso dias == 366, criando um loop infinito travando os aparelhos.
- Java (Break Rotulado): Permite associar uma etiqueta (label) a um laço externo para que o comando break consiga interromper múltiplos laços aninhados de uma só vez.
- JavaScript (for...in vs for...of): O for...in percorre as chaves/índices (retornando Strings), enquanto o for...of percorre os valores reais da coleção.
- Pós-teste (do-while): Executa o bloco de código obrigatoriamente ao menos uma vez antes de validar a condição de parada.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;          // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. · §8.4 · p. 352-353 · exemplos/09_desvio_protegido

Compreendi que o uso desenfreado do desvio incondicional (goto) prejudica a legibilidade do software, gerando bugs graves e dificultando a verificação do fluxo de execução.

Nota: A controvérsia do goto e desvios estruturados:

- Dijkstra (1968): Publicou o artigo clássico "Go To Statement Considered Harmful", defendendo que o goto torna a execução do código caótica e imprevisível.
- A Falha "goto fail": Em C, um segundo goto fail; duplicado sem bloco de chaves fez o programa desviar sempre para o final da função, ignorando a verificação de assinatura digital e aceitando certificados falsos.
- Linguagens Atuais: O Java omitiu a instrução goto. Linguagens como C, C++ e Go mantêm o goto, mas recomendam o uso de desvios restritos e seguros (break, continue, return).

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. · §8.5 · p. 353-355 · exemplos/09_desvio_protegido

Compreendi a teoria dos comandos protegidos de Dijkstra, onde guardas condicionais condicionam o fluxo e introduzem o não-determinismo, servindo de fundação para concorrência moderna como o select em Go.

Nota: Conceito e aplicações práticas de Guardas:

- Estrutura com Guardas: Avalia uma série de condições (guardas). Se mais de uma for verdadeira, o algoritmo escolhe qualquer um dos ramos de forma não-determinística. Se nenhuma for verdadeira, encerra com erro.
- Base do modelo CSP (Hoare, 1978): O não-determinismo é essencial para lidar com concorrência e troca de mensagens assíncronas.
- Go (select): O comando select em Go atua como um herdeiro direto dessa teoria: quando múltiplos canais de comunicação estão prontos ao mesmo tempo, o Go sorteia aleatoriamente qual deles processará.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Compreendi as soluções para os problemas práticos apresentados no primeiro conjunto de exercícios de busca de bugs em JavaScript e Python.

Nota: Análise detalhada de diagnóstico:

1. JavaScript:

- (a) Saída: "Total: 0012".
- (b) Causa: O for...in itera sobre as chaves/índices do array como Strings ("0", "1", "2"), concatenando-os ao valor inicial.
- (c) Correção: Trocar por for (const p of precos), obtendo a soma numérica correta (60).

1. Python:

- (a) Saída: 180.0.
- (b) Causa: Passar 0 no parâmetro de desconto faz com que a expressão 0 or 10 avalie 0 como falso, aplicando indevidamente o valor padrão de 10.
- (c) Correção: Usar checagem explícita de valor nulo: d = 10 if d is None else d, retornando 200.0.

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Compreendi os erros de sintaxe e controle de fluxo do segundo conjunto de exercícios em Java e C.

Nota: Análise detalhada de diagnóstico: 3. Java:

- (a) Saída: "inválido".
- (b) Causa: Falta do comando break nos blocos case, fazendo a execução cair em cascata até o default.
- (c) Correção: Adicionar break; ao fim de cada instrução de case.

1. C:

- (a) Saída: "saldo insuficiente" e "fim" em linhas separadas.
- (b) Causa: Ponto e vírgula colocado logo após a condição do if (if (saque > saldo);), criando um corpo condicional vazio e tornando a instrução de impressão incondicional.
- (c) Correção: Remover o ; e agrupar o bloco sob chaves {}.

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Compreendi as armadilhas de escopo em Go e de divisão inteira em Java demonstradas no terceiro bloco de exercícios.

Nota: Análise detalhada de diagnóstico: 5. Go:

- (a) Saída: "dentro: 20" e "fora: 10".
- (b) Causa: O uso do operador de declaração curta `:=` dentro do bloco do `if` criou uma nova variável local `x` que ocultou (shadowing) a variável externa.
- (c) Correção: Utilizar atribuição simples `x = x * 2` para atualizar a variável externa.

1. Java:

- (a) Saída: "0.0%".
- (b) Causa: Como ambos os operandos (`acertos` e `total`) são inteiros, a operação `7 / 10` realiza uma divisão inteira truncada resultando em 0, sendo convertida para 0.0 apenas na atribuição.
- (c) Correção: Aplicar cast explícito antes da operação: `(double) acertos / total * 100`.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : gotos restritos

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

Compreendi o resumo geral da aula: os erros mais comuns de programação surgem de detalhes das regras das linguagens, e o uso de boas práticas e linguagens com tipagem/estruturas rígidas ajuda a evitá-los.

Nota: Sintaxe comparativa das armadilhas e mitigação oferecida pelas linguagens:

- Precedência: Prevenida por uso de parênteses e sintaxes estritas.
- Efeitos Colaterais: Mitigados em linguagens como Java que fixam a ordem de avaliação e pela preferência por funções puras.
- Conversões Implícitas: Evitadas por compiladores como Go e Rust, que exigem conversão explícita de tipos.
- Atribuição Indevida em Condicionais: Eliminada em linguagens que tratam atribuição como instrução isolada (Python, Go, Rust).
- Seleção/Iteração Insegura: Resolvidas através de construções como `match` exaustivo, blocos de chaves obrigatórios e iteradores dedicados (`for...of`).